

Anatomy of a Song Map

One JSON file per recording that says **what the song does and when**. Something else turns it into a light show. The file is the product.

01 THE IDEA



The first 24 seconds of *Levels* as the file records it. Thin ticks are beats, thick lines are bar starts, bands are named parts. Everything is seconds from zero — never bars, never samples.

02 THE ONE RULE

The map may not contain a lighting word

BELONGS IN THE FILE	NEVER IN THE FILE	WHY
drop at 19.911	strobe at 19.911	A drone show reads the same file. "Strobe" means nothing to 120 drones; "drop" does.
energy 0.92	brightness 92%	The file describes the song. What to do about it is the reader's decision, not the map's.

And the second rule: never write a number you did not measure

`null` is a legitimate answer and worth more than a guess — once a guess is written down, nobody can tell it from a measurement. Every value carries how it was obtained:

- MEASURED
- A HUMAN SAID SO
- A GUESS, QUARANTINED

03 THE CONTENTS

What one file holds

Real values from `maps/model/levels.map.json` — 776 KB for a 237-second song.

FIELD	LEVELS	WHAT IT SAYS
grid	128.0 BPM	Tempo, and where beat one sits (0.223 s). Everything else hangs off this.

FIELD	LEVELS	WHAT IT SAYS
beats	506	Every beat, in seconds.
downbeats	127	Which of those start a bar.
chapters	17	Named parts: intro, verse, break, drop, outro.
moments	16	Six kinds only — build, drop, stop, quiet, spotlight, return. Adding a seventh is refused.
energy	127	How big it feels, once per bar.
accents	3 520	Individual hits, each labelled with the instrument it came from.
chords	125	One per bar. C#m , bass note F# .
melody	213	The sung line, note by note.
notes	2 540	Every note the transcriber found, on every instrument.
vectors	759 KB	A number-vector per beat from a music model. Lets a reader ask "which two parts sound alike" without knowing why.

Plus 28 further observations — key (**E major**), sidechain depth (**0.147**), stereo width, where the voice is silent, swing, and so on. Anything a reader might not recognise lives here, and a reader that does not recognise a field ignores it.

04 HOW IT IS BUILT

Audio in, file out

One command per song: **bash tools/pipeline.sh levels** . Fifteen steps, three Python environments, because the libraries genuinely conflict.

01	ear.py — find the tempo, the beats, the parts, the energy	STDLIB
02	separate.py — split into 6 instrument tracks, get the hits	DEMUCS
03	deep.py — mood, novelty, lyrics	MUQ • WHISPER
04	barphase.py — which beat of the four starts a bar	MUST RUN BEFORE 06
05	moments.py — put each drop where the sound actually steps	MUST RUN BEFORE 06
06	derive.py — phrase length, and cross-checks between fields	STDLIB
07	chordmini.py — chords, from a model trained on human annotations	GPU • 3 S/SONG
08	notes.py → melody.py — transcribe every instrument, take the top voice as the tune	PYTHON 3.11 ONLY

09	attack.py — move every hit to where it physically starts	LIBROSA
10	vectors.py — per-beat embeddings	MERT

A step whose environment is missing is **skipped with a note**, not failed. With only the standard library you still get a file that validates and scores.

MODEL	WRITES	WHAT IT IS FOR
htdemucs_6s	accents, stems	Splits the mix into 6 instrument tracks, so a hit is attributed to a drum rather than guessed from the mix.
MuQ-large	novelty	Frame embeddings, cosine distance across a bar either side. Finds <i>where the song changes</i> without being told what a section is — used to cross-check the named parts.
MuQ-MuLan-large	mood	Music and text in one embedding space, so a section can be scored against words like <i>euphoric</i> directly.
MERT-v1-95M	vectors	One vector per beat. Lets a reader find parts that sound alike with no idea why they do.
ChordMini BTC	chords	170-chord vocabulary, trained on human annotations. Beat the hand-written estimate outright.
basic-pitch	notes, melody	Polyphonic transcription, per instrument.
Whisper	lyrics	Words and their timings.
beat_this	nothing	A neural beat tracker, used only to <i>disagree</i> with us. It confirmed our grids on three songs; on the other two its own beats scored worse than ours, so it was not adopted.

05 HOW WE KNOW IT IS RIGHT

Ten checks, each one reads the audio

There is no answer key. Every check re-measures the recording and asks whether the file agrees — so there is nothing to leak and nothing to memorise.

CHECK	WEIGHT	LEVELS	WHAT IT ACTUALLY MEASURES
grid	3.0	0.97	Is the kick loud where the file says the beats are?
bars	2.0	1.00	Do the named parts start on bar lines?
moments	2.0	0.86	Is each drop within half a beat of the real loudness step?
downbeats	1.5	not scorable	Is the bar start louder than the other beats? On four-on-the-floor, nothing is.

CHECK	WEIGHT	LEVELS	WHAT IT ACTUALLY MEASURES
sections	1.5	1.00 	Does the sound change at a boundary more than at 64 sets of random times?
energy	1.5	0.99 	Correlation against how loud the record actually is.
accents	1.0	0.79 	Is each claimed hit on a real onset peak?
chords	1.0	0.79 	Do the claimed notes carry the energy? Capped: the true chords score this too.
melody	1.0	1.00 	Is each claimed pitch the loudest one sounding there?
pump	0.5	1.00 	Claims +0.147; recording says +0.148.

A change to the scorer must make a broken file score worse

Two files exist purely to be wrong — one with every beat half a beat late, one built from guesses. Both must read **0.00**. `tools/verify.sh` asserts it. Three of my changes today passed that test; one did not, and was reverted.

06 THE BOARD

Where the five songs stand

SONG	START OF DAY	NOW	NEXT BEST MAP
levels	0.75	0.94 	0.70 musicstate
starlight	0.44	0.89 	—
dont-look-down	—	0.89 	—
mizhiyoram	—	0.83 	—
the-nights	0.70	0.83 	0.65 baseline

the-nights was never tuned against. The pipeline ran on it once, start to finish, and it came out top of that board — which is the only evidence that the other four are not overfitted.

06B THE GAP

What a show needs that no check rewards

Every field above is **retrospective** — it says what is true at a time, never what is coming. A reader asked for a frame at 18.0 s cannot know a drop is 1.9 s away without walking the moment list itself, so every reader reinvents anticipation and none of them agree. These six are not scored by anything.

FIELD	LEVELS SAYS	WHY IT HAS TO BE IN THE FILE
anticipation	0.5–2.5 bars of warning	Per bar: how many bars to the next event and what kind, how far through a build we are. A ramp has to start <i>before</i> the thing it is ramping into.
surprise	peak at 137.1 s	How far this bar departs from what the four before it predicted. A bar identical to its neighbours needs nothing however loud it is; a bar that breaks the pattern is the whole point. Not the same as novelty — novelty finds boundaries, this asks whether the bar was <i>predictable</i> , and they part company by the third repeat of a section.
lead	drums 38 bars, other 35, vocals 29	Which stem is <i>carrying</i> the bar. The file said how loud each one is and never which to point at.
arc	peaks 18% in	Where the record peaks, as a fraction of its length. Starlight and Don't Look Down peak at 90%. A show that spends everything on the first drop has nowhere left to go.
meter	4 beats / bar	<code>beats_per_bar=4</code> was a <i>default argument</i> nothing had ever tested, and every downbeat and chapter in the repo rests on it. Now measured, from the lag at which the drum pattern repeats.
tempo_stability	–14.1 ms over 237 s	The grid is one period and one phase, exact for a DAW and wrong for anything played by people. The kicks drift 0.008 BPM across the whole record, so the rigid grid is justified — and now there is a number instead of an assumption.

Three wrong turns inside one field, kept in the file

`stems.sources` is *normalised per stem*, so ranking those six series against each other is meaningless — each is scaled to 1.0 at its own loudest bar, and Starlight read as piano-led for 43 bars from a stem 20 dB down. Rescaling by each stem's true level fixed it. Then a median-based presence test cut the vocal hook out of Levels, because a chopped sample is silent most of the record and **intermittent is not absent**.

And one thing left alone, because it is somebody else's door

`spotlight` is one of the six legal moment kinds — a **lighting word inside the tier whose entire purpose is to contain none**. The validator errors on "beam" or "lamp" in a data position and hardcodes "spotlight" as legal. A drone reader cannot spotlight. The musical fact is that one voice is exposed; *spotlight* names the response to it.

What moved, and why

FIX	EFFECT	THE ACTUAL BUG
drop times	+2 beats early	Drops were snapped to bar lines. The drops in <i>Levels</i> land on half bars, so every one was pushed 0.94 s late — the rig sat in the break through the loudest moment of the song. Found from the stage, not from a number.
bar phase	-1 beat	On <i>mizhiyoram</i> the bar started on the <i>quietest</i> beat of the four. Hidden because the parts were positioned from the bar line, so "the parts agree with the bar line" was arithmetic, not evidence.
hit timing	-7 to -17 ms	Every hit was late. The detectors that find hits are known to run late, and the check I validated them against was the same kind of detector. Two late instruments agreeing proves nothing. Re-measured against the raw waveform.
coverage	90% → 100%	The file was losing 10% because a check <i>could not</i> grade one field — not because the field was missing. Charging a map for the genre of its song.
sections check	cost us 0.05	It compared boundaries against <i>one</i> random draw. On one song it could not tell real boundaries from ones rolled 61 seconds. Now 64 draws — and two of our scores went down, correctly.
the board page	no-store	No cache headers anywhere, so the leaderboard was served from browser cache. Every number above was already true for hours; the screen showed old ones.

08 THE TRAP

Never grade a thing against its own output

This is the mistake the project keeps paying for, in three different disguises.

LOOKED LIKE PROOF	WAS ACTUALLY
Beat grid agreed with the tracker to 9 ms.	Same features on both sides. Real error was 75 ms; a human ear found it.
Named parts all sat on bar lines.	The parts were <i>generated</i> at the bar line. Hid a wrong bar line for a day.
Hit times matched onset peaks in the mix.	Both sides were the same kind of late detector. Everything was 10 ms late.

The counter-measure is cheap: before believing a number, ask what the two sides of the comparison have in common. If it is code or features, the number is decoration.

Limelight · **REPRODUCE.md** for a fresh clone · **tools/verify.sh** for everything that can say "broken"

All figures measured from **maps/model/*.map.json** and **listen/mapeval.py** .